

Mestrado em Engenharia Informática

Unidade Curricular: Segurança e Privacidade dos Dados

Relatório de Implementação de uma Infraestrutura de Chaves Públicas (PKI)

Diogo Vieira | 1240448

João Monteiro | 1240469

Abril, 2025

Índice

Introdução.....	3
Diagrama da PKI	4
Fluxo	5
Menu	5
Criar Utilizador	5
Revogar Certificado Utilizador.....	8
Renovar Certificado Utilizador	8
Renovar Certificado Válido	9
Reemitir Certificado Inválido	10
Diretório do Utilizador Depois das Modificações	12
Testar Acesso	12
Pelo menu:.....	13
Pelo Browser:.....	15
Listar Utilizadores.....	17
Validade Certificados.....	18
Server/Aplicação	18
Estrutura e Caminhos dos Diretórios	19
Criação da Root CA	22
Script: setup_root_ca.sh.....	22
Script: openssl.cnf	23
Criação da Intermediate CA	27
Script: setup_intermediate_ca.sh	27
Script: openssl.cnf	29
Servidor	32
Script: certificates_server.sh	32
Ficheiro de configuração do NGINX: /etc/nginx/sites-available/pki-app	33
Aplicação (app.py).....	35
Utilizadores	36
Criação de novo utilizador: certificates_users.sh	36
Conexão do Utilizador	38
Revogar um Certificado de um Utilizador: revoke_users.sh.....	38
Script: concat_crls.sh.....	39
Renovar um Certificado de um Utilizador: renew_users.sh.....	41
Reemitir um Certificado de um Utilizador: reissue_revoked_users.sh	43
Referências.....	45

Introdução

Este projeto tem como objetivo a criação de uma infraestrutura de chaves públicas (PKI), desde a geração e gestão de certificados digitais até à sua validação em interface web, com autenticação mútua baseada em certificados (mTLS). O foco está na aplicação de boas práticas de segurança, com base em criptografia assimétrica e na utilização do OpenSSL como ferramenta principal.

Ao longo deste trabalho, são implementadas as seguintes componentes:

- Uma Root Certificate Authority (CA), que atua como a autoridade principal e confiável da organização.
- Uma CA Intermédia, responsável pela emissão dos certificados de utilizadores e serviços.
- Um servidor web (NGINX) configurado para autenticação mTLS.
- Um conjunto de scripts para automatizar a criação, revogação, renovação e reemissão de certificados.
- Uma aplicação web backend (Flask) protegida por autenticação baseada em certificados.

Com esta estrutura, a comunicação entre clientes e servidor é assegurada através de criptografia forte, com validação de identidade baseada em certificados digitais, promovendo um modelo de segurança robusto e escalável.

Diagrama da PKI

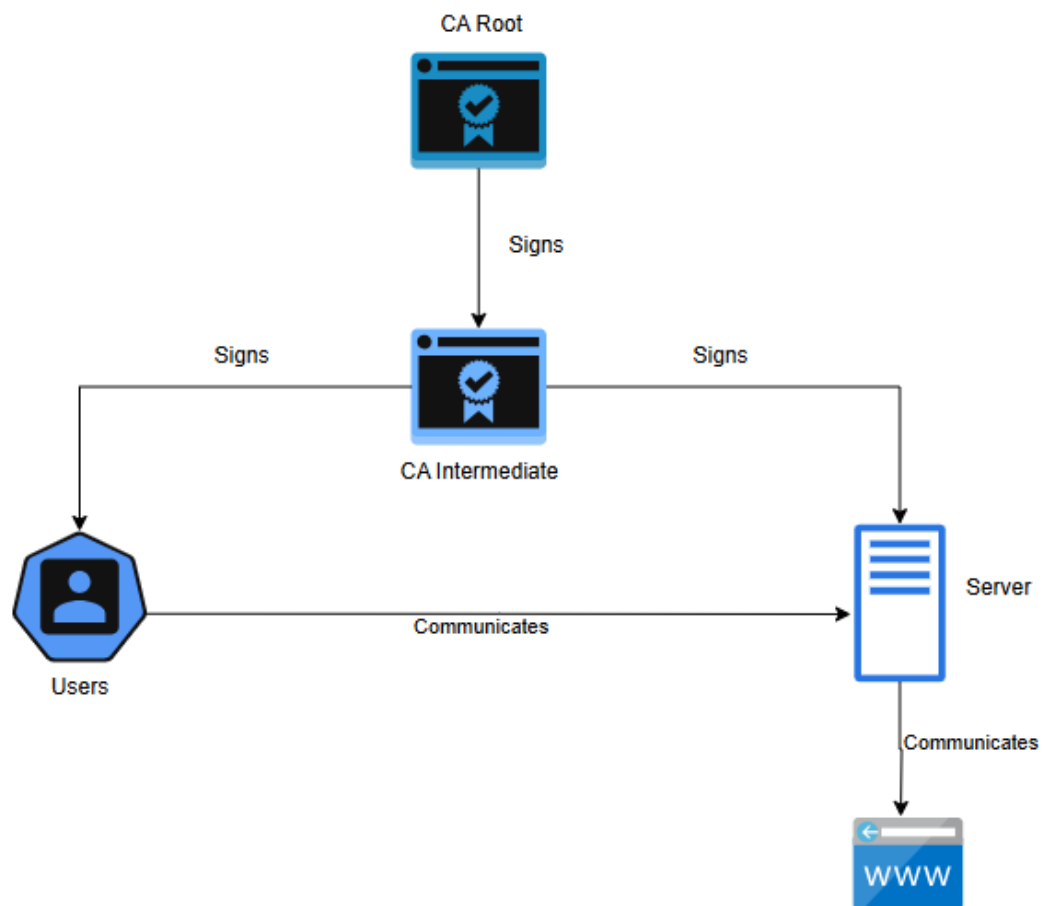


Figura 1 - Diagrama

Fluxo

Menu

Ao entrar no diretório do projeto é só fazer `./menu.sh`, e o ficheiro `menu.sh` tem várias opções para realizar ações.

```
==== 🛡️ PKI MENU - Segurança e Privacidade dos Dados ====
1) Criar novo utilizador
2) Revogar certificado de utilizador
3) Renovar/Reemitir certificado de utilizador
4) Testar acesso (curl)
5) Listar utilizadores existentes
6) Ver validade de um certificado de utilizador
7) Sair

Escolha uma opção [1-7]: █
```

Figura 2 – menu

Criar Utilizador

Para criar o utilizador podemos ir à pasta onde está o script **`certificates_user.sh`** e meter `./certificates_user.sh` (nome do utilizador), mas para criar de forma mais intuitiva basta escolher a opção 1 do menu.

```

===== PKI MENU - Segurança e Privacidade dos Dados ===== enter address
1) Criar novo utilizador
2) Revogar certificado de utilizador
3) Renovar/Reemitir certificado de utilizador
4) Testar acesso (curl)
5) Listar utilizadores existentes
6) Ver validade de um certificado de utilizador
7) Sair

Escolha uma opção [1-7]: 1
Nome do novo utilizador: xpto
A gerar chave privada para xpto ...
A gerar CSR para xpto ...
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.

Country Name (2 letter code) [AU]:PT
State or Province Name (full name) [Some-State]:Porto
Locality Name (eg, city) []:Porto
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Isep
Organizational Unit Name (eg, section) []:Isep-xpto
Common Name (e.g. server FQDN or YOUR name) []:Xpto
Email Address []:xpto@gmail.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:kali
An optional company name []:Isep
A assinar o certificado com a Intermediate CA ...
Using configuration from /home/kali/SEGPRD-Project1/PKI/intermediate/openssl.
cnf
Enter pass phrase for /home/kali/SEGPRD-Project1/PKI/intermediate/private/int
ermediate.key.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
    Serial Number: 4116 (0x1014)
    Validity
        Not Before: Apr 13 13:42:46 2025 GMT
        Not After : Apr 23 13:42:46 2026 GMT
    Subject:
        countryName           = PT
        stateOrProvinceName   = Porto
        localityName          = Porto
        organizationName       = Isep
        organizationalUnitName = Isep-xpto
        commonName             = Xpto
        emailAddress           = xpto@gmail.com
    X509v3 extensions:
        X509v3 Basic Constraints:

```

Figura 3 - Criar Utilizador

```
X509v3 BASIC CONSTRAINTS:
    CA:FALSE
Netscape Cert Type:
    SSL Client, S/MIME
Netscape Comment:
    OpenSSL Generated Client Certificate
X509v3 Subject Key Identifier:
    7C:A6:D6:CD:75:99:89:47:7F:C6:86:51:25:14:E7:AE:7A:9D:89:58
X509v3 Authority Key Identifier:
    9A:FD:BA:BD:B5:AC:4D:08:9B:B5:3F:95:B7:A2:76:01:F7:24:47:F7
X509v3 Key Usage: critical
    Digital Signature, Non Repudiation, Key Encipherment
X509v3 Extended Key Usage:
    TLS Web Client Authentication, E-mail Protection
Certificate is to be certified until Apr 23 13:42:46 2026 GMT (375 days)

Write out database with 1 new entries
Database updated
Enter Export Password:
Verifying - Enter Export Password:
🚀 Certificado criado com sucesso para xpto em:
    - Chave privada: /home/kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.key
    .pem
    - Certificado: /home/kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.cer
    t.pem
    - PKCS#12: /home/kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.p12
Pressiona Enter para continuar...
```

Figura 4 - Criar Utilizador (continuação)

Ao criar o utilizador se formos à sua pasta podemos ver os ficheiros criados:

```
(root@kali)-[/home/.../PKI/users/users/xpto]
# ls -l
total 20
-r--r--r-- 1 root root 1809 Apr 13 14:42 xpto.cert.pem
-rw-r--r-- 1 root root 1090 Apr 13 14:42 xpto.csr.pem
-r----- 1 root root 1704 Apr 13 14:41 xpto.key.pem
-r--r--r-- 1 root root 6003 Apr 13 14:42 xpto.p12
```

Figura 5 - Ficheiros da criação do user

Revogar Certificado Utilizador

Para revogar um certificado basta ir ao diretório do script **revoke_users.sh** e meter **./revoke_users.sh** (nome do utilizador), mas para revogar de forma mais intuitiva basta escolher a opção 2 do menu.

```
==== 🛡️ PKI MENU - Segurança e Privacidade dos Dados ====
1) Criar novo utilizador
2) Revogar certificado de utilizador
3) Renovar/Reemitir certificado de utilizador
4) Testar acesso (curl)
5) Listar utilizadores existentes
6) Ver validade de um certificado de utilizador
7) Sair

Escolha uma opção [1-7]: 2
👤 Nome do utilizador a revogar: xpto
🔴 A revogar o certificado de xpto...
Using configuration from /home/kali/SEGPRD-Project1/PKI/intermediate/openssl.cnf
Revoking Certificate 1014.
Database updated
📁 A gerar nova CRL da Intermédia...
Using configuration from /home/kali/SEGPRD-Project1/PKI/intermediate/openssl.cnf
📁 A gerar nova CRL da Root...
Using configuration from /home/kali/SEGPRD-Project1/PKI/root/openssl.cnf
🔗 A concatenar CRLs (Root + Intermédia) em /home/kali/SEGPRD-Project1/PKI/intermediate/crl/full-chain.crl.pem
🔧 A reiniciar o NGINX...
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
✔ Certificado revogado e NGINX atualizado com nova CRL!

Pressiona Enter para continuar...
```

Figura 6 - Revogar Certificado Utilizador

Renovar Certificado Utilizador

Para renovar um certificado, temos duas opções, uma para renovar um certificado de um utilizador com o certificado válido e outra opção de reemitir um certificado de um utilizador com o certificado inválido, e para isso basta ir ao diretório do script **renew_users.sh** e meter **./renew_users.sh** (nome do utilizador) ou ao diretório do script **reissue_revoked_users.sh** e meter **./reissue_revoked_users.sh** (nome do utilizador), respetivamente, mas para revogar de forma mais intuitiva basta escolher a opção 3 do menu.

Depois ao escolher a opção do menu aparece outra aba para escolher qual das duas opções pretende.

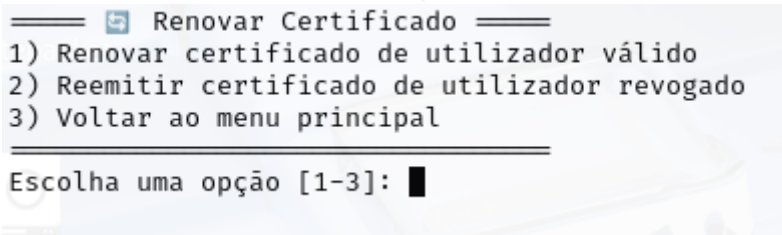


Figura 7 - Escolher opção de renovar

Renovar Certificado Válido

Escolher a opção 1:

```
Escolha uma opção [1-3]: 1
Nome do utilizador a renovar: xpto
A gerar novo CSR para xpto com chave antiga...
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.

Country Name (2 letter code) [AU]:PT
State or Province Name (full name) [Some-State]:Porto
Locality Name (eg, city) []:Porto
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Isep
Organizational Unit Name (eg, section) []:Isep-xpto
Common Name (e.g. server FQDN or YOUR name) []:Xpto
Email Address []:xpto@gmail.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:kali
An optional company name []:kali
A assinar novo certificado com a Intermediate CA...
Using configuration from /home/kali/SEGPRD-Project1/PKI/intermediate/openssl.
cnf
Enter pass phrase for /home/kali/SEGPRD-Project1/PKI/intermediate/private/int
ermediate.key.pem:
Check that the request matches the signature
Signature ok
ERROR:There is already a certificate for /C=PT/ST=Porto/L=Porto/O=Isep/OU=Ise
p-xpto/CN=Xpto/emailAddress=xpto@gmail.com
The matching entry has the following details
Type :Valid
Expires on :260423142425Z
Serial Number :1015
File name :unknown
Subject Name :/C=PT/ST=Porto/L=Porto/O=Isep/OU=Isep-xpto/CN=Xpto/emailAdres
s=xpto@gmail.com
chmod: cannot access '/home/kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.ce
rt.renovado.pem': No such file or directory
A gerar novo ficheiro PKCS#12 (.p12)...
Could not open file or uri for loading certificates from -in file from /home/
kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.cert.renovado.pem: No such fil
e or directory
chmod: cannot access '/home/kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.re
novado.p12': No such file or directory
Certificado renovado com sucesso!
Ficheiros atualizados:
- Novo certificado: /home/kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.ce
rt.renovado.pem
- Novo ficheiro .p12: /home/kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.
renovado.p12
```

Figura 8 - Renovar Certificado Utilizador

Reemitir Certificado Inválido

Escolher a opção 2:

```
===== 📠 Renovar Certificado =====
1) Renovar certificado de utilizador válido
2) Reemitir certificado de utilizador revogado
3) Voltar ao menu principal
=====

Escolha uma opção [1-3]: 2
👤 Nome do utilizador a reemitir (revogado): xpto
📄 O certificado antigo foi revogado. A proceder à reemissão ...
✍️ A gerar novo CSR com a chave existente ...
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.

-----
Country Name (2 letter code) [AU]:PT
State or Province Name (full name) [Some-State]:Porto
Locality Name (eg, city) []:Porto
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Isep
Organizational Unit Name (eg, section) []:Isep-xpto
Common Name (e.g. server FQDN or YOUR name) []:Xpto
Email Address []:xpto@gmail.com

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:kali
An optional company name []:Isep
✍️ A assinar novo certificado com a CA Intermédia ...
Using configuration from /home/kali/SEGPRD-Project1/PKI/intermediate/openssl.
cnf
Check that the request matches the signature
Signature ok
```

Figura 9 - Reemitir Certificado do Utilizador

```

Certificate Details:
  Serial Number: 4117 (0x1015)
  Validity
    Not Before: Apr 13 14:24:25 2025 GMT
    Not After : Apr 23 14:24:25 2026 GMT
  Subject:
    countryName           = PT
    stateOrProvinceName   = Porto
    localityName          = Porto
    organizationName       = Isep
    organizationalUnitName = Isep-xpto
    commonName             = Xpto
    emailAddress           = xpto@gmail.com
  X509v3 extensions:
    X509v3 Basic Constraints:
      CA:FALSE
    Netscape Cert Type:
      SSL Client, S/MIME
    Netscape Comment:
      OpenSSL Generated Client Certificate
    X509v3 Subject Key Identifier:
      7C:A6:D6:CD:75:99:89:47:7F:C6:86:51:25:14:E7:AE:7A:9D:89:58
    X509v3 Authority Key Identifier:
      9A:FD:BA:BD:B5:AC:4D:08:9B:B5:3F:95:B7:A2:76:01:F7:24:47:F7
    X509v3 Key Usage: critical
      Digital Signature, Non Repudiation, Key Encipherment
    X509v3 Extended Key Usage:
      TLS Web Client Authentication, E-mail Protection
Certificate is to be certified until Apr 23 14:24:25 2026 GMT (375 days)

Write out database with 1 new entries
Database updated
👉 A gerar novo ficheiro PKCS#12 (.p12) ...
Enter Export Password:
Verifying - Enter Export Password:
👉 Reemissão completa para xpto
  - Novo certificado: /home/kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.cert.pem
  - Novo .p12:       /home/kali/SEGPRD-Project1/PKI/users/users/xpto/xpto.p12
2
  - Antigo certificado guardado como: xpto.cert.revogado.pem

```

Figura 10 - Reemitir Certificado do Utilizador (continuação)

Diretório do Utilizador Depois das Modificações

```
(root@kali)-[/home/.../PKI/users/users/xpto]
# ls -ls CTRL+C to quit
total 52
-r--r--r-- 1 root root 1809 Apr 13 15:24 xpto.cert.pem
-r--r--r-- 1 root root 1809 Apr 13 15:24 xpto.cert.reissue.pem
-r--r--r-- 1 root root 1809 Apr 13 14:42 xpto.cert.revogado.pem
-rw-r--r-- 1 root root 1090 Apr 13 15:24 xpto.csr.reissue.pem
-rw-r--r-- 1 root root 1090 Apr 13 15:28 xpto.csr.renovado.pem
-rw-r--r-- 1 root root 1090 Apr 13 14:42 xpto.csr.revogado.pem
-r----- 1 root root 1704 Apr 13 14:41 xpto.key.pem
-r--r--r-- 1 root root 6003 Apr 13 15:24 xpto.p12
-r--r--r-- 1 root root 6003 Apr 13 15:24 xpto.reissue.p12
-r--r--r-- 1 root root 6003 Apr 13 14:42 xpto.revogado.p12
```

Figura 11 - Ficheiros do user depois das modificações

Testar Acesso

Para testar se os certificados estão válidos ou não tenho duas opções, a do menu, a partir de um curl ou indo diretamente ao browser.

Pelo menu:

Opção 4 do menu:

Certificado Válido:

```

===== PKI MENU - Segurança e Privacidade dos Dados =====
1) Criar novo utilizador
2) Revogar certificado de utilizador
3) Renovar/Reemitir certificado de utilizador
4) Testar acesso (curl)
5) Listar utilizadores existentes
6) Ver validade de um certificado de utilizador
7) Sair

Escolha uma opção [1-7]: 4
% Nome do utilizador a testar (com certificado válido): xpto
* A testar acesso via curl...
* Host servidor:443 was resolved.
* IPv6: (none)
* IPv4: 127.0.0.1
* Trying 127.0.0.1:443...
* GnuTLS priority: NORMAL:-ARCFOUR-128:-CTYPE-ALL:+CTYPE-X509:-VERS-SSL3.0
* found 2 certificates in /home/kali/SEGPRD-Project1/PKI/intermediate/certs/ca-chain.cert.pem
* found 458 certificates in /etc/ssl/certs
* ALPN: curl offers h2,http/1.1
* SSL connection using TLS1.3 / ECDHE_RSA_AES_256_GCM_SHA384
* server certificate verification OK
* server certificate status verification SKIPPED
* common name: servidor (matched)
* server certificate expiration date OK
* server certificate activation date OK
* certificate public key: RSA
* certificate version: #3
* subject: CN=servidor
* start date: Sun, 06 Apr 2025 01:14:40 GMT
* expire date: Thu, 16 Apr 2026 01:14:40 GMT
* issuer: C=PT,ST=Porto,O=Isep,CN=Isep Intermediate CA
* ALPN: server accepted http/1.1
* Connected to servidor (127.0.0.1) port 443
* using HTTP/1.x
> GET / HTTP/1.1
> Host: servidor
> User-Agent: curl/8.13.0-rc3
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx
< Date: Sun, 13 Apr 2025 13:44:47 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 107
< Connection: keep-alive
<
👋 Bem-vindo, certificado:

* Connection #0 to host servidor left intact
emailAddress=xpto@gmail.com,CN=Xpto,OU=Isep-xpto,O=Isep,L=Porto,ST=Porto,C=PT
Pressiona Enter para continuar...

```

Figura 12 - certificado válido (curl)

Certificado Inválido:

```
Escolha uma opção [1-7]: 4
% Nome do utilizador a testar (com certificado válido): xpto
🚀 A testar acesso via curl ...
* Host servidor:443 was resolved.
* IPv6: (none)
* IPv4: 127.0.0.1
* Trying 127.0.0.1:443 ...
* GnuTLS priority: NORMAL:-ARCFOUR-128:-CTYPE-ALL:+CTYPE-X509:-VERS-SSL3.0
* found 2 certificates in /home/kali/SEGPRD-Project1/PKI/intermediate/certs/ca-chain.cert.pem
* found 458 certificates in /etc/ssl/certs
* ALPN: curl offers h2,http/1.1
* SSL connection using TLS1.3 / ECDHE_RSA_AES_256_GCM_SHA384
* server certificate verification OK
* server certificate status verification SKIPPED
* common name: servidor (matched)
* server certificate expiration date OK
* server certificate activation date OK
* certificate public key: RSA
* certificate version: #3
* subject: CN=servidor
* start date: Sun, 06 Apr 2025 01:14:40 GMT
* expire date: Thu, 16 Apr 2026 01:14:40 GMT
* issuer: C=PT,ST=Porto,O=Isep,CN=Isep Intermediate CA
* ALPN: server accepted http/1.1
* Connected to servidor (127.0.0.1) port 443
* using HTTP/1.x
> GET / HTTP/1.1
> Host: servidor
> User-Agent: curl/8.13.0-rc3
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 400 Bad Request
< Server: nginx
< Date: Sun, 13 Apr 2025 14:05:06 GMT
< Content-Type: text/html
< Content-Length: 208
< Connection: close
<
<html>
<head><title>400 The SSL certificate error</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<center>The SSL certificate error</center>
<hr><center>nginx</center>
</body>
</html>
* shutting down connection #0
```

Figura 13 - Certificado Inválido (curl)

Pelo Browser:

1. Acedemos pelo browser ao link <https://servidor>:

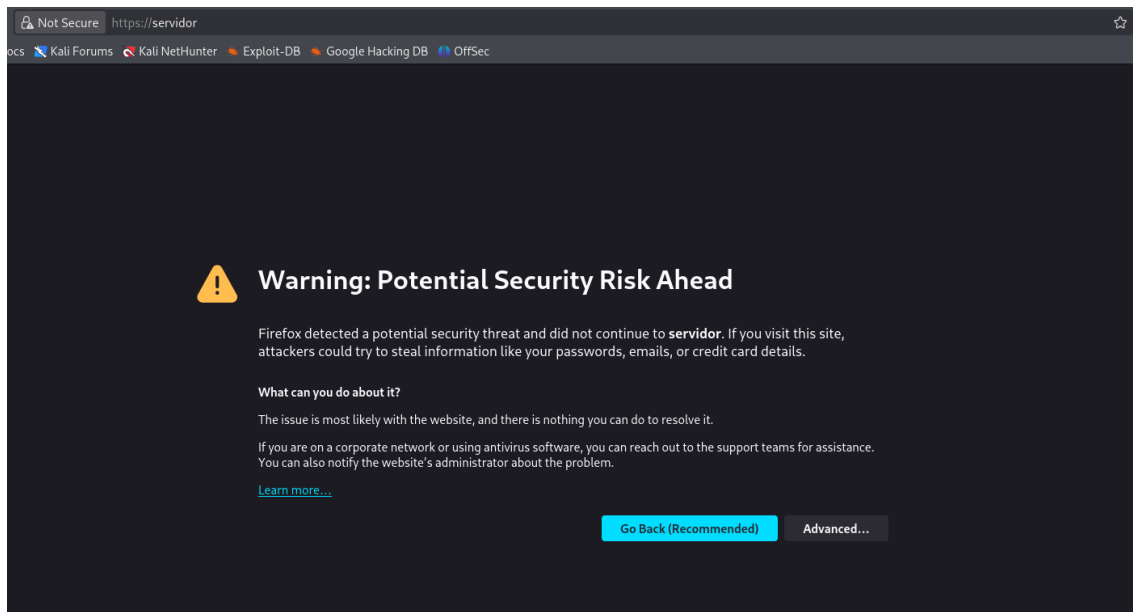


Figura 14 - Aceder ao link pelo browser

2. Ir às definições para colocar o certificado

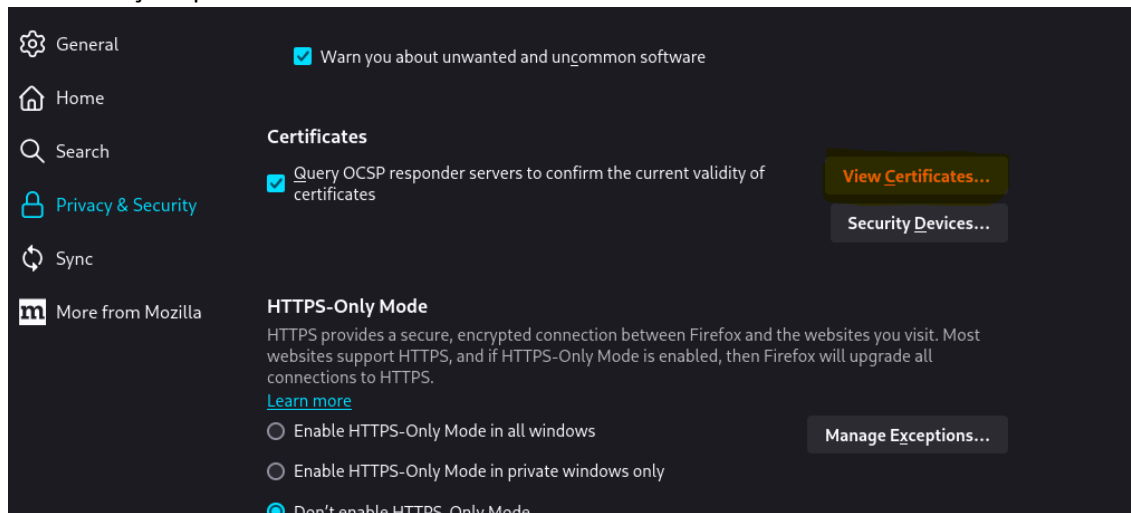


Figura 15 – Definições

3. Importar certificado

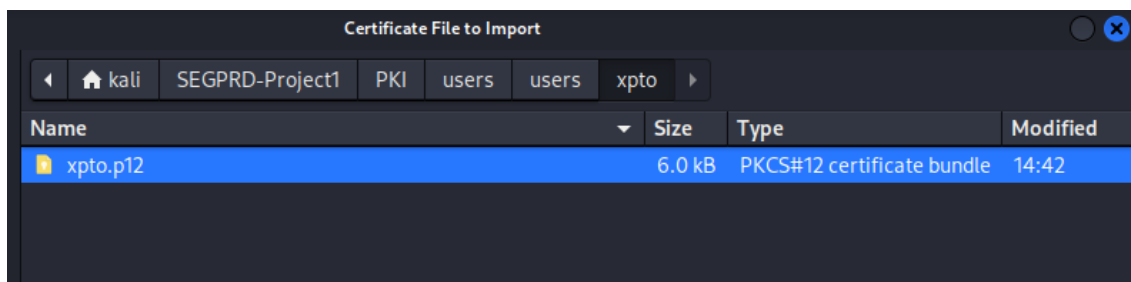


Figura 16 - Inserir Certificado

Ao clicar no ficheiro .p12 é necessário dar a password, anteriormente definida.

4. Voltar ao link e confirmar o certificado

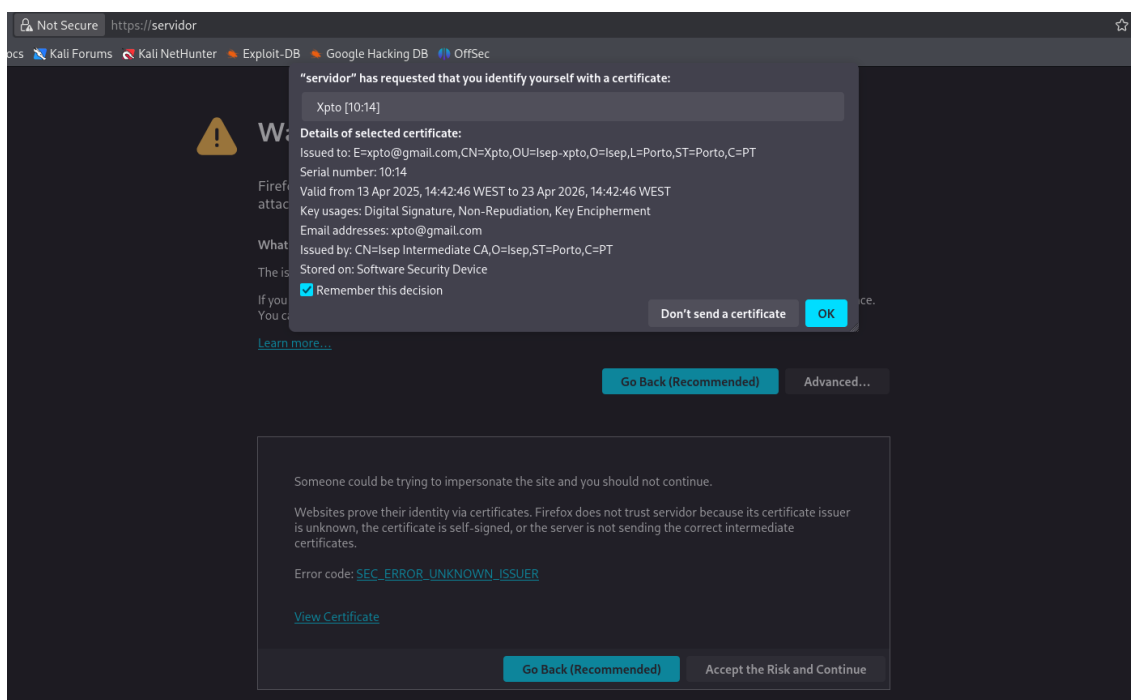


Figura 17 - Confirmar Certificado

5. Certificado válido

Caso o certificado seja válido deverá aparecer isto:

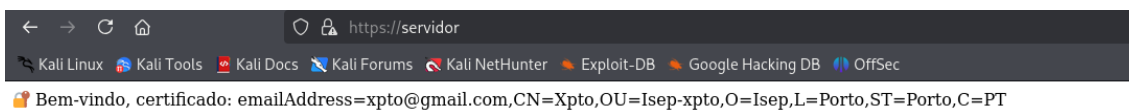


Figura 18 - Certificado Válido

6. Certificado Inválido

Caso o certificado seja inválido deverá aparecer isto:

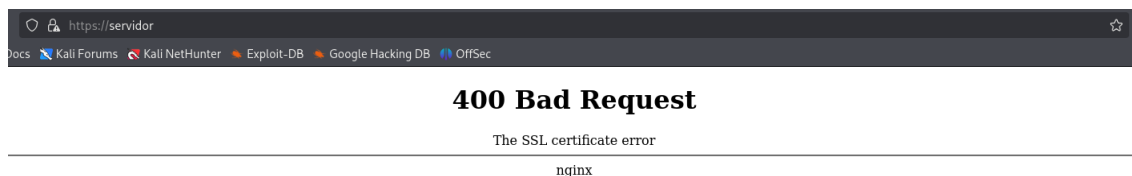


Figura 19 - Certificado Inválido

Listar Utilizadores

Na opção 5 do menu:

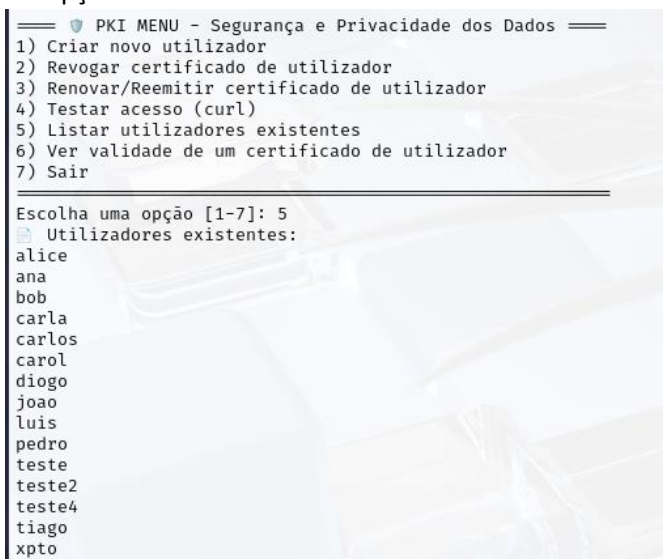


Figura 20 - Lista de utilizadores

Validade Certificados

Opção 6 do menu:

```
==== PKI MENU - Segurança e Privacidade dos Dados ====
1) Criar novo utilizador
2) Revogar certificado de utilizador
3) Renovar/Reemitir certificado de utilizador
4) Testar acesso (curl)
5) Listar utilizadores existentes
6) Ver validade de um certificado de utilizador
7) Sair

Escolha uma opção [1-7]: 6
Nome do utilizador: xpto
Data de expiração do certificado de xpto:
notAfter=Apr 23 14:24:25 2026 GMT
```

Figura 21 - Validade do Certificado do Utilizador

Server/Aplicação

Para criar o certificado do server podemos ir à pasta onde está o script **certificates_server.sh** e meter `./certificates_server.sh`.

Para a aplicação é só ir ao diretório onde está o ficheiro **app.py** e correr da seguinte maneira:

```
(venv)-(root@kali)-[/home/kali/SEGPRD-Project1/flask-app]
# sudo python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [13/Apr/2025 14:37:08] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [13/Apr/2025 14:44:47] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [13/Apr/2025 14:45:48] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [13/Apr/2025 14:45:49] "GET /favicon.ico HTTP/1.0" 404 -
127.0.0.1 - - [13/Apr/2025 14:46:24] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [13/Apr/2025 14:46:26] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [13/Apr/2025 14:50:48] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [13/Apr/2025 15:31:22] "GET / HTTP/1.0" 200 -
127.0.0.1 - - [13/Apr/2025 15:31:30] "GET / HTTP/1.0" 200 -
```

Figura 22 - Correr app e logs

Na Figura 22 para além de vermos a aplicação a correr também vemos os logs de requisição ao backend, a aplicação Flask está funcional e a receber pedidos do NGINX.

Estrutura e Caminhos dos Diretórios

```
(root@kali)-[/home/kali/SEGPRD-Project1]
# ls -l
total 28
-rwxr-xr-x 1 root root 632 Apr 9 20:49 concat_crls.sh
drwxr-xr-x 4 root root 4096 Apr 8 17:31 flask-app
-rwxr-xr-x 1 root root 2162 Apr 9 21:41 menu.sh
drwxr-xr-x 6 root root 4096 Apr 9 17:16 PKI
-rwxr-xr-x 1 root root 1449 Apr 6 18:31 setup_intermediate_ca.sh
-rwxr-xr-x 1 root root 639 Apr 5 15:20 setup_root_ca.sh
drwxr-xr-x 5 root root 4096 Apr 6 17:48 venv
```

Figura 23 - Base do projeto

```
(root@kali)-[/home/kali/SEGPRD-Project1/PKI]
# ls -l
total 16
drwxr-xr-x 7 root root 4096 Apr 10 16:36 intermediate
drwxr-xr-x 6 root root 4096 Apr 9 20:42 root
drwxr-xr-x 2 root root 4096 Apr 9 17:41 servers
drwxr-xr-x 3 root root 4096 Apr 9 21:29 users
```

Figura 24 - Diretório da PKI

```
(root@kali)-[/home/kali/SEGPRD-Project1/PKI/root]
# ls -l
total 36
drwxr-xr-x 2 root root 4096 Apr 5 15:33 certs
drwxr-xr-x 2 root root 4096 Apr 9 20:42 crl
-rw-r--r-- 1 root root 76 Apr 5 15:34 index.txt
-rw-r--r-- 1 root root 21 Apr 5 15:34 index.txt.attr
-rw-r--r-- 1 root root 0 Apr 5 15:31 index.txt.old
drwxr-xr-x 2 root root 4096 Apr 5 15:34 newcerts
-rw-r--r-- 1 root root 1642 Apr 9 20:42 openssl.cnf
drwxr-xr-x 2 root root 4096 Apr 5 15:21 private
-rw-r--r-- 1 root root 5 Apr 5 15:34 serial
-rw-r--r-- 1 root root 5 Apr 5 15:31 serial.old
```

Figura 25 - Diretório da CA Root

```
(root@kali)-[/home/kali/SEGPRD-Project1/PKI/intermediate]
# ls -l
total 56
drwxr-xr-x 2 root root 4096 Apr  5 15:34 certs
drwxr-xr-x 2 root root 4096 Apr  9 20:50 crl
-rw-r--r-- 1 root root  5 Apr 10 16:36 crlnumber
-rw-r--r-- 1 root root  5 Apr  9 21:46 crlnumber.old
drwxr-xr-x 2 root root 4096 Apr  5 15:34 csr
-rw-r--r-- 1 root root 2355 Apr 10 16:36 index.txt
-rw-r--r-- 1 root root  21 Apr 10 16:36 index.txt.attr
-rw-r--r-- 1 root root  21 Apr 10 16:36 index.txt.attr.old
-rw-r--r-- 1 root root 2240 Apr 10 16:36 index.txt.old
drwxr-xr-x 2 root root 4096 Apr 10 16:36 newcerts
-rw-r--r-- 1 root root 2234 Apr  9 20:12 openssl.cnf
drwxr-xr-x 2 root root 4096 Apr  5 15:30 private
-rw-r--r-- 1 root root  5 Apr 10 16:36 serial
-rw-r--r-- 1 root root  5 Apr 10 16:33 serial.old
```

Figura 26 - Diretório da CA Intermediate

```
(root@kali)-[/home/kali/SEGPRD-Project1/PKI/servers]
# ls -l
total 16
-rwxr-xr-x 1 root root 750 Apr  6 02:14 certificates_server.sh
-r--r--r-- 1 root root 1651 Apr  6 02:14 servidor.cert.pem
-rw-r--r-- 1 root root 891 Apr  6 02:14 servidor.csr.pem
-r----- 1 root root 1700 Apr  6 02:14 servidor.key.pem
```

Figura 27 - Diretório do Server

```
(root@kali)-[/home/kali/SEGPRD-Project1/PKI/users]
# ls -l
total 20
-rwxr-xr-x 1 root root 1413 Apr  9 21:04 certificates_users.sh
-rwxr-xr-x 1 root root 2606 Apr  9 21:29 reissue_revoked_users.sh
-rwxr-xr-x 1 root root 1381 Apr  9 21:12 renew_users.sh
-rwxr-xr-x 1 root root 1478 Apr  9 21:02 revoke_users.sh
drwxr-xr-x 16 root root 4096 Apr 10 16:32 users
```

Figura 28 - Diretório dos Users

```
(root@kali)-[/home/.../SEGPRD-Project1/PKI/users/users]
# ls -l
total 56
drwxr-xr-x 2 root root 4096 Apr  9 21:15 alice
drwxr-xr-x 2 root root 4096 Apr  9 21:45 ana
drwxr-xr-x 2 root root 4096 Apr  5 23:49 bob
drwxr-xr-x 2 root root 4096 Apr  9 19:01 carla
drwxr-xr-x 2 root root 4096 Apr  6 18:43 carlos
drwxr-xr-x 2 root root 4096 Apr  5 23:51 carol
drwxr-xr-x 2 root root 4096 Apr  9 17:20 diogo
drwxr-xr-x 2 root root 4096 Apr  6 02:49 joao
drwxr-xr-x 2 root root 4096 Apr  8 22:09 luis
drwxr-xr-x 2 root root 4096 Apr  6 04:20 pedro
drwxr-xr-x 2 root root 4096 Apr  9 21:21 teste
drwxr-xr-x 2 root root 4096 Apr  9 21:47 teste2
drwxr-xr-x 2 root root 4096 Apr 10 16:36 teste4
drwxr-xr-x 2 root root 4096 Apr  6 18:41 tiago
```

Figura 29 - Diretório onde são alocados os users criados

```
(root@kali)-[/home/.../PKI/users/users/pedro]
# ls -l
total 20
-r--r--r-- 1 root root 1814 Apr  6 04:20 pedro.cert.pem
-rw-r--r-- 1 root root 1098 Apr  6 04:20 pedro.csr.pem
-r----- 1 root root 1704 Apr  6 04:20 pedro.key.pem
-rw----- 1 root root 6019 Apr  6 04:20 pedro.p12
```

Figura 30 - Exemplo da pasta de um User criado

```
(root@kali)-[/home/kali/SEGPRD-Project1/flask-app]
# ls -l
total 12
-rwxr-xr-x 1 root root 420 Apr  6 03:28 app.py
drwxr-xr-x 2 root root 4096 Apr  6 04:44 __pycache__
drwxr-xr-x 5 root root 4096 Apr  6 01:41 venv
```

Figura 31 - Aplicação (Flask)

Criação da Root CA

A Root CA é a entidade raiz da infraestrutura PKI. Opera de forma offline e a sua única função é assinar o certificado da CA Intermédia. Isto garante a sua segurança, protegendo-a de ataques diretos.

Ela atua como o “pai de todos os certificados”, sendo a origem de confiança para todos os certificados emitidos - direta ou indiretamente.

Para a criação do Root CA fizemos um script **setup_root_ca.sh**, a partir deste script criamos a estrutura e organização dos ficheiros do PKI e assinamos o Intermediate CA, para assinar chamamos ainda o ficheiro **openssl.cnf**.

Script: setup_root_ca.sh

Este script implementa os seguintes passos:

1. Cria a estrutura de diretórios da Root CA:

- `mkdir -p $ROOT_CA_DIR/{certs,crl,newcerts,private}`
- `touch $ROOT_CA_DIR/index.txt`
- `echo 1000 > $ROOT_CA_DIR/serial`

Descrição:

- `certs/`: onde será guardado o certificado da Root CA
- `crl/`: onde ficam listas de revogação (CRL) - se for necessário gerar
- `newcerts/`: onde ficarão certificados emitidos (não aplicável à Root)
- `private/`: chave privada da Root CA (deve ser protegida!)
- `index.txt`: registo de todos os certificados assinados (neste caso, só a CA Intermédia)
- `serial`: número de série do próximo certificado a ser assinado (começa em 1000)

2. Gera a chave privada da Root CA com encriptação AES-256:

- `openssl genrsa -aes256 -out $ROOT_CA_DIR/private/ca.key.pem 4096`
- `chmod 400 $ROOT_CA_DIR/private/ca.key.pem`

Descrição:

- Chave privada RSA com 4096 bits, protegida por AES-256
- Guarda-se em local seguro, com permissões restritas

3. Gera o certificado da Root CA com extensões v3_ca:

- `openssl req -config $ROOT_CA_DIR/openssl.cnf \`
- `-key $ROOT_CA_DIR/private/ca.key.pem \`
- `-new -x509 -days 7300 -sha256 -extensions v3_ca \`
- `-out $ROOT_CA_DIR/certs/ca.cert.pem`
- `chmod 444 $ROOT_CA_DIR/certs/ca.cert.pem`

Descrição:

- O certificado é autoassinado (a própria Root CA assina o seu certificado)
- Usa as extensões v3_ca definidas no openssl.cnf
- Validade de 7300 dias (aproximadamente 20 anos)
- Leitura apenas — proteção contra alterações acidentais

Script: openssl.cnf

Este ficheiro indica ao OpenSSL **como deve comportar-se quando cria, assina ou revoga certificados**. Cada bloco [secção] define parâmetros para um determinado contexto.

1. Configuração padrão da CA

- [ca]
- default_ca = CA_default

Descrição:

- Define que a CA principal vai usar a configuração que está em [CA_default].

2. Diretórios e ficheiros essenciais

- [CA_default]
- dir = /home/kali/SEGPRD-Project1/PKI/root
- certs = \$dir/certs
- crl_dir = \$dir/crl
- database = \$dir/index.txt
- new_certs_dir = \$dir/newcerts
- certificate = \$dir/certs/ca.cert.pem
- serial = \$dir/serial
- private_key = \$dir/private/ca.key.pem

Descrição:

- dir: diretório base da Root CA
- certs: onde vai guardar os certificados emitidos (ex: da CA Intermédia)
- crl_dir: onde serão guardadas listas de revogação (CRL)
- database: registo de certificados emitidos (índice)
- new_certs_dir: onde ficam os ficheiros .pem dos certificados emitidos
- certificate: caminho do certificado da própria Root CA
- private_key: caminho da chave privada usada para assinar

3. Validades

- default_days = 375
- default_crl_days = 30

Descrição:

- default_days: validade padrão dos certificados emitidos
- default_crl_days: número de dias até expirar uma CRL
- Mesmo que a Root só assine a Intermédia, ainda precisa de CRL se for preciso revogá-la.

4. Assinatura e políticas

- default_md = sha256
- preserve = no
- policy = policy_strict

Descrição:

- default_md: algoritmo hash SHA-256
- preserve: define se mantém ou não os campos do CSR (neste caso, não)
- policy: define regras de validação para o CSR — ver [policy_strict]

5. Define o que o OpenSSL exige num CSR antes de assinar:

- [policy_strict]
- countryName = match
- stateOrProvinceName = match
- organizationName = match
- organizationalUnitName = optional
- commonName = supplied
- emailAddress = optional

Descrição:

- match – tem de ser igual
- optional – não é obrigatório existir
- supplied – tem de estar presente, mas pode ser diferente

6. Criação do certificado da Root

- [req]
- default_bits = 4096
- distinguished_name = req_distinguished_name
- string_mask = utf8only
- default_md = sha256
- x509_extensions = v3_ca

Descrição:

- Criação de certificados autoassinados (como o da Root)
- Usa as extensões v3_ca

7. Campos esperados

- [req_distinguished_name]
- countryName = Country Name (2 letter code)
- stateOrProvinceName = State or Province Name
- localityName = Locality Name
- organizationName = Organization Name
- commonName = Common Name

8. Extensões para Root CA

- [v3_ca]
- subjectKeyIdentifier = hash
- authorityKeyIdentifier = keyid:always,issuer
- basicConstraints = critical, CA:true
- keyUsage = critical, digitalSignature, cRLSign, keyCertSign

Descrição:

- Criação de certificados autoassinados (como o da Root)
- Usa as extensões v3_ca

9. Assinar a CA Intermédia

- [v3_intermediate_ca]
- basicConstraints = critical, CA:true, pathlen:0
- keyUsage = critical, digitalSignature, cRLSign, keyCertSign
- subjectKeyIdentifier = hash
- authorityKeyIdentifier = keyid:always,issuer

Descrição:

- pathlen:0: essa Intermédia não pode criar outra CA abaixo dela
- Estas são extensões usadas quando a Root assina a CA Intermédia.

10. Extensões usadas na criação da CRL

- authorityKeyIdentifier = keyid:always

Criação da Intermediate CA

A CA Intermédia é responsável por emitir todos os certificados, sejam eles de utilizadores ou servidores, ela também gere e mantém CRLs e não pode criar outra CA abaixo dela (impedido por pathlen:0).

O seu certificado é assinado pela Root CA.

Script: `setup_intermediate_ca.sh`

Este script implementa os seguintes passos:

1. Cria a estrutura de diretórios:

- `mkdir -p $INT_CA_DIR/{certs,crl,csr,newcerts,private}`
- `touch $INT_CA_DIR/index.txt`
- `echo 1000 > $INT_CA_DIR/serial`
- `echo 1000 > $INT_CA_DIR/crlnumber`

Descrição:

- semelhante ao do `setup_root_ca`
- `crlnumber`: contador das CRLs emitidas

2. Gera a chave privada da CA Intermédia:

- `openssl genrsa -aes256 -out $INT_CA_DIR/private/intermediate.key.pem 4096`

Descrição:

- Gera uma chave RSA com 4096 bits e encriptação AES-256

3. Gera o CSR:

- `openssl req -config $INT_CA_DIR/openssl.cnf \`
- `-new -sha256 \`
- `-key $INT_CA_DIR/private/intermediate.key.pem \`
- `-out $INT_CA_DIR/csr/intermediate.csr.pem`

Descrição:

- Cria um CSR com base na chave privada
- Vai ser assinado pela Root CA para se tornar o certificado da CA Intermédia

4. A Root CA assina este CSR:

- `openssl ca -config $ROOT_CA_DIR/openssl.cnf \`
- `-extensions v3_intermediate_ca \`
- `-days 3650 -notext -md sha256`
- `-in $INT_CA_DIR/csr/intermediate.csr.pem \`
- `-out $INT_CA_DIR/certs/intermediate.cert.pem`

Descrição:

- Aqui a Root CA entra em ação:
 - Usa o ficheiro `openssl.cnf` da Root
 - Aplica as extensões `v3_intermediate_ca`
 - Cria o certificado da Intermédia com validade de 10 anos (3650 dias)

5. Verificação Final

- `if [ -f $INT_CA_DIR/certs/intermediate.cert.pem ]; then`
- `echo "[✓] Certificado da Intermediate CA criado com sucesso."`
- `else`
- `echo "[X] Algo correu mal. O certificado não foi criado."`
- `fi`

6. Criação o ficheiro `ca-chain.cert.pem`:

- `cat $INT_CA_DIR/certs/intermediate.cert.pem \`
- `$ROOT_CA_DIR/certs/ca.cert.pem > $INT_CA_DIR/certs/ca-chain.cert.pem`

Descrição:

- Junta o certificado da Intermédia e da Root num ficheiro só
- Este ficheiro vai ser necessário para que:
 - Os navegadores validem a cadeia de confiança
 - O NGINX verifique os certificados dos utilizadores
 - O .p12 dos utilizadores inclua a cadeia correta

Script: openssl.cnf

Este ficheiro define as secções que o OpenSSL usa para executar ações como emitir certificados, gerar CRLs e criar chaves/CSRs.

Muito semelhante ao openssl.cnf do root mas com algumas diferenças.

1. CRL e assinaturas

- `crl` = `$dir/crl/intermediate.crl.pem`
- `crl_extensions` = `crl_ext`
- `default_crl_days` = 30

Descrição:

- Caminho da CRL da Intermédia
- Expira a cada 30 dias por padrão
- Usa as extensões definidas em `[crl_ext]`

2. Outras configurações

- `default_md` = `sha256`
- `default_days` = 375
- `preserve` = `no`
- `policy` = `policy_loose`

Descrição:

- Hash usado: SHA-256
- Validade padrão de certificados: 375 dias
- Política de emissão: `policy_loose` (mais permissiva que a da Root)

3. Política de CSR

- `[ policy_loose ]`
- `countryName` = optional
- `stateOrProvinceName` = optional
- `localityName` = optional
- `organizationName` = optional
- `organizationalUnitName` = optional
- `commonName` = supplied
- `emailAddress` = optional

Descrição:

- Só exige o campo Common Name (CN) — os restantes são opcionais
- Isto dá flexibilidade para emitir certificados para vários contextos (users, servidores)

4. Para criar CSRs

- [req]
- default_bits = 4096
- distinguished_name = req_distinguished_name
- string_mask = utf8only
- default_md = sha256
- x509_extensions = v3_intermediate_ca

Descrição:

- Define os parâmetros de criação de CSRs e autoassinatura
- Usa extensões v3_intermediate_ca quando cria o certificado da própria CA Intermédia

5. Campos esperados

- [req_distinguished_name] - Campos pedidos quando crias o CSR com openssl req.

6. Extensões aplicadas ao certificada da própria CA Intermediate

- basicConstraints = critical, CA:true, pathlen:0
- keyUsage = critical, digitalSignature, cRLSign, keyCertSign
- subjectKeyIdentifier = hash
- authorityKeyIdentifier = keyid:always,issuer

Descrição:

- Torna-a uma CA
- Pode emitir certificados e CRLs
- Não pode criar outra CA abaixo dela (pathlen:0)
- Contém identificadores para validação da cadeia

7. Para os certificados de utilizador

- basicConstraints = CA:FALSE
- nsCertType = client, email
- nsComment = "OpenSSL Generated Client Certificate"
- subjectKeyIdentifier = hash
- authorityKeyIdentifier = keyid,issuer
- keyUsage = critical, nonRepudiation, digitalSignature, keyEncipherment
- extendedKeyUsage = clientAuth, emailProtection

Descrição:

- Define que não é uma CA
- Pode ser usado para:
 - Autenticação do cliente (clientAuth)
 - Assinatura digital
 - Encriptação

8. Para os certificados de server

- basicConstraints = CA:FALSE
- nsCertType = server
- nsComment = "OpenSSL Generated Server Certificate"
- subjectKeyIdentifier = hash
- authorityKeyIdentifier = keyid,issuer
- keyUsage = critical, digitalSignature, keyEncipherment
- extendedKeyUsage = serverAuth

Descrição:

- Também não é uma CA
- Pode ser usado para servidores (ex: NGINX com TLS)

9. Extensões da CRL

- authorityKeyIdentifier=keyid:always,issuer

Descrição:

- Ajuda na validação da origem da CRL — garante que foi emitida pela entidade certa

Servidor

O servidor utiliza um certificado assinado pela CA Intermédia e está configurado com NGINX para requerer autenticação mTLS.

Através do qual é possível fazer o processo de criação, assinatura e validação do certificado do servidor.

Script: `certificates_server.sh`

Este script implementa os seguintes passos:

1. Gera chave privada:

- `openssl genrsa -out "$SERVER_DIR/$SERVER_NAME.key.pem" 2048`
- `chmod 400 "$SERVER_DIR/$SERVER_NAME.key.pem"`

Descrição:

- Cria uma chave privada RSA de 2048 bits para o servidor.
- É guardada com permissões restritas (apenas leitura para o owner).

2. Geração do CSR

- `openssl req -new -key "$SERVER_DIR/$SERVER_NAME.key.pem" \`
- `-out "$SERVER_DIR/$SERVER_NAME.csr.pem" \`
- `-subj "/CN=$SERVER_NAME"`

Descrição:

- Cria um pedido de assinatura de certificado, com o Common Name (CN) definido como "servidor".
- Este CSR será assinado pela CA Intermédia.

3. Assinatura com a CA Intermédia

- `openssl ca -config openssl.cnf \`
- `-extensions server_cert -days 375 -notext -md sha256 \`
- `-in "$SERVER_DIR/$SERVER_NAME.csr.pem" \`
- `-out "$SERVER_DIR/$SERVER_NAME.cert.pem"`

Descrição:

- A CA Intermédia assina o CSR.
- O certificado é emitido com validade de 375 dias.
- Aplica as extensões `server_cert`, definidas no `openssl.cnf` da Intermédia: (`extendedKeyUsage = serverAuth`)
- Ou seja, este certificado é reconhecido como válido para autenticação de servidores (ex: HTTPS via NGINX).

Ficheiro de configuração do NGINX: /etc/nginx/sites-available/pki-app

O objetivo aqui é garantir:

- I. Que o NGINX apresenta o certificado do servidor
- II. Que exige aos clientes um certificado válido
- III. Que valida a cadeia de confiança e as revogações (CRL)

1. Certificados do lado do servidor

- `ssl_certificate /home/kali/SEGPRD-Project1/PKI/servers/servidor.cert.pem;`
- `ssl_certificate_key /home/kali/SEGPRD-Project1/PKI/servers/servidor.key.pem;`

Descrição:

- Aqui o NGINX usa o certificado gerado no script anterior para identificar-se como servidor.

2. Verificação de certificado do cliente

- `ssl_client_certificate /home/kali/SEGPRD-Project1/PKI/intermediate/certs/ca-chain.cert.pem;`
- `ssl_verify_client on;`
- `ssl_verify_depth 2;`

Descrição:

- O NGINX exige um certificado do cliente (utilizador).
- Verifica se o certificado apresentado está assinado por uma CA válida (Root ou Intermédia), até 2 níveis acima.
- Usa o ficheiro `ca-chain.cert.pem`, que contém:
 - `intermediate.cert.pem`
 - `root.cert.pem`

3. Verificação de revogação (CRL)

- `ssl_crl /home/kali/SEGPRD-Project1/PKI/intermediate/crl/full-chain.crl.pem;`

Descrição:

- O NGINX usa uma CRL combinada da Root e da Intermédia (gerada com `concat_crls.sh`).

- Se o certificado do cliente estiver revogado, o acesso é negado.

4. Segurança do TLS

- `ssl_protocols TLSv1.2 TLSv1.3;`
- `ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384';`
- `ssl_prefer_server_ciphers on;`

Descrição:

- O Apenas são permitidos protocolos e algoritmos seguros e modernos.
- O servidor impõe a escolha do cifrão.

5. Comunicação com o backend (Flask)

- `location / {`
- `proxy_pass http://127.0.0.1:5000;`
- `proxy_set_header Host $host;`
- `proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;`
-
- `# Header com DN do certificado do cliente`
- `proxy_set_header X-SSL-CERT $ssl_client_s_dn;`
- `proxy_set_header X-SSL-VERIFY $ssl_client_verify;`
- `}`

Descrição:

- O NGINX está configurado para atuar como um gateway TLS entre o cliente e a aplicação Flask.
- Depois de validar o cliente, o NGINX envia cabeçalhos ao backend com:
 - A identidade (Distinguished Name) do certificado
 - Se a verificação foi bem-sucedida

Aplicação (app.py)

Este é o **último elo da cadeia de confiança**, onde o servidor trata as credenciais do cliente e mostra o resultado.

O funcionamento do backend é através do Flask no ficheiro app.py, tendo em conta a configuração do NGINX e o sistema de autenticação via mTLS (mutual TLS).

Este script implementa os seguintes passos:

1. Função index

- `cert = request.headers.get('X-SSL-CERT')`
- `verify = request.headers.get('X-SSL-VERIFY')`

Descrição:

- O Flask lê os headers HTTP enviados pelo NGINX.
- Estes headers são definidos no NGINX com:
 - `proxy_set_header X-SSL-CERT $ssl_client_s_dn;`
 - `proxy_set_header X-SSL-VERIFY $ssl_client_verify;`
- X-SSL-CERT → contém o **Distinguished Name (DN)** do certificado do cliente.
- X-SSL-VERIFY → diz se a verificação foi bem-sucedida (SUCCESS), ou falhou (FAILED).

2. Validar o Processo

- `if verify == "SUCCESS" and cert:`
- `return f"🔒 Bem-vindo, certificado:\n\n{cert}"`
- `return "❌ Acesso negado. Certificado não encontrado."`

Descrição:

- Se o certificado do cliente for válido e verificado, o servidor mostra uma mensagem de boas-vindas com o conteúdo do DN.
- Caso contrário, nega o acesso com uma mensagem de erro.

3. Correr a App

- `app.run(host='127.0.0.1', port=5000)`

Descrição:

- Isto corre a aplicação Flask localmente na porta 5000.
- O NGINX é quem ouve na porta 443 (HTTPS) e encaminha o tráfego internamente para este backend Flask via `proxy_pass`.

4. Ligação com mTLS

A app não faz autenticação direta, pois confia que o NGINX fica responsável por essa parte.

O backend recebe os headers com a confirmação e identidade do utilizador.

Utilizadores

Criação de novo utilizador: certificates_users.sh

O certificado do Utilizador é um ficheiro emitido por uma CA (a CA Intermediate), contém a identidade do utilizador, está assinado digitalmente pela CA e será usado para autenticação forte através do mTLS.

Este script implementa os seguintes passos:

1. Diretórios na criação do utilizador

- USER_NAME=\$1
- USER_DIR="\$BASE_DIR/users/users/\$USER_NAME"
- mkdir -p "\$USER_DIR"

Descrição:

- Irá guardar os ficheiros do utilizador dentro de uma pasta isolada com o nome de cada utilizador e irá conter a chave, o CSR, o certificado e o .p12.

2. Gerar chave privada

- openssl genrsa -out "\$USER_DIR/\$USER_NAME.key.pem" 2048
- chmod 400 "\$USER_DIR/\$USER_NAME.key.pem"

Descrição:

- Cria uma chave privada de 2048 bits.
- Esta chave nunca deve sair da máquina do utilizador — é usada para provar a identidade (daí o chmod 400).

3. Gerar o CSR

- openssl req -new -key "\$USER_DIR/\$USER_NAME.key.pem" \
- -out "\$USER_DIR/\$USER_NAME.csr.pem"

Descrição:

- O CSR é um pedido formal à CA Intermédia para obter um certificado.
- Contém a chave pública e as informações do utilizador.

4. Assinatura da CA Intermédia

- openssl ca -batch -config "\$INT_CA_DIR/openssl.cnf" \
- -extensions usr_cert \
- -days 375 -notext -md sha256 \
- -in "\$USER_DIR/\$USER_NAME.csr.pem" \
- -out "\$USER_DIR/\$USER_NAME.cert.pem"

Descrição:

- A CA Intermédia assina o CSR e emite um certificado válido por 375 dias.
- Usa as extensões definidas na secção [usr_cert] do openssl.cnf:
 - extendedKeyUsage = clientAuth : Ou seja, o certificado é válido para autenticação do cliente.

5. Exportar para o ficheiro .p12

- openssl pkcs12 -export \
- -inkey "\$USER_DIR/\$USER_NAME.key.pem" \
- -in "\$USER_DIR/\$USER_NAME.cert.pem" \
- -certfile "\$INT_CA_DIR/certs/ca-chain.cert.pem" \
- -out "\$USER_DIR/\$USER_NAME.p12"
- chmod 444 "\$USER_DIR/\$USER_NAME.p12"

Descrição:

- Junta o certificado do utilizador, a sua chave privada e a cadeia de confiança (Intermediate + Root).
- O ficheiro .p12 pode ser **importado no browser** ou usado em ferramentas como curl. Para ter uma experiência mais dinâmica, damos permissões ao ficheiro .p12 para os ficheiros possam ser usados mas sem serem modificados acidentalmente.

Conexão do Utilizador

Como o utilizador tem conexão:

Utilização no mTLS

O utilizador apresenta este certificado ao NGINX durante a conexão segura:

- `curl -v https://servidor \`
- `--cert alice.cert.pem \`
- `--key alice.key.pem \`
- `--cacert ca-chain.cert.pem`

- O NGINX valida o certificado contra a cadeia e CRL.

- Se for válido, reencaminha a informação para o backend Flask (via cabeçalhos HTTP)

Revogar um Certificado de um Utilizador: `revoke_users.sh`

Revogar um certificado antes da sua data de expiração é essencial em casos de comprometimento da chave privada de um utilizador, colaborador que saiu da empresa ou certificado usado de forma indevida.

A revogação é feita pela CA que emitiu o certificado, neste caso, a CA Intermédia, e publicada numa CRL.

Este script implementa os seguintes passos:

1. Revogar um certificado

- `openssl ca -config "$INT_CA_DIR/openssl.cnf" -revoke "$CERT_FILE" -passin pass:kali`

Descrição:

- Este comando revoga o certificado na base de dados da CA Intermédia.
- É adicionado ao ficheiro `index.txt` com a flag R (revogado).

2. Atualizar CRLs

São atualizadas tanto na Intermediate como no Root

- `openssl ca -config "$INT_CA_DIR/openssl.cnf" -gencrl -out "$INT_CA_DIR/crl/intermediate.crl.pem" -passin pass:kali`
- `openssl ca -config "$ROOT_CA_DIR/openssl.cnf" -gencrl -out "$ROOT_CA_DIR/crl/root.crl.pem" -passin pass:kali`

3. Concatenar CRLs

- `cat "$ROOT_CA_DIR/crl/root.crl.pem" "$INT_CA_DIR/crl/intermediate.crl.pem" > "$FULL_CHAIN_CRL"`

Descrição:

- O NGINX só aceita uma única CRL, por isso é necessário combinar a da Root com a da Intermédia.

4. Reload do Nginx

- `sudo nginx -t && sudo systemctl reload nginx`

Descrição:

- Verifica a sintaxe da configuração (garante que não tem erros)
- Atualiza o NGINX para que use a nova CRL

Nesta parte de revogar tivemos algumas dificuldades em implementar, pois no Nginx apenas estávamos a dar a lista de revogados do CA Intermediate, e quando tínhamos esse comando todos os certificados apareciam como revogados na conexão com o server, quando retirávamos a linha de código todos os certificados apareciam como validos.

Então adicionamos também a CRL da CA Root, mas apercebemo-nos que o Nginx apenas aceitava um ficheiro de CRL. Como alternativa, inicialmente, criámos o ficheiro **concat_crls.sh**.

Script: [concat_crls.sh](#)

Este script implementa os seguintes passos:

1. Definir caminhos

- `ROOT_CRL="PKI/root/crl/root.crl.pem"`
- `INTER_CRL="PKI/intermediate/crl/intermediate.crl.pem"`
- `OUT_CRL="PKI/intermediate/crl/full-chain.crl.pem"`

Descrição:

- `ROOT_CRL`: ficheiro com as revogações feitas pela Root
- `INTER_CRL`: revogações da CA Intermédia
- `OUT_CRL`: ficheiro de saída com as duas juntas

2. Verificar a existência das CRLs

- `if [[ ! -f "$ROOT_CRL" || ! -f "$INTER_CRL" ]]; then`
- `echo "✗ Uma das CRLs não existe. Verifica as paths:"`
- `echo "Root: $ROOT_CRL"`
- `echo "Intermediate: $INTER_CRL"`
- `exit 1`
- `Fi`

Descrição:

- Confirma se ambos os ficheiros existem
- Evita erro de concatenação se uma das CRLs ainda não tiver sido gerada

3. Concatenar CRLs

- `cat "$ROOT_CRL" "$INTER_CRL" > "$OUT_CRL"`

Descrição:

- Junta as duas CRLs num único ficheiro.
- A ordem importa: primeiro a CRL da Root, depois a da Intermédia. (Pois o NGINX verifica a cadeia de validação e precisa de conseguir verificar revogações em todas as CAs da cadeia, num único ficheiro)

Renovar um Certificado de um Utilizador: renew_users.sh

Renovar um certificado significa gerar um novo certificado com os mesmos dados, mas com nova validade, sem revogar o anterior.

Por norma um certificado é renovado quando o certificado atual está próximo da data de expiração ou para continuar a autenticação sem precisar criar um utilizador novo.

Este script implementa os seguintes passos:

1. Variáveis e Caminhos

- `SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"`
- `PROJECT_ROOT="$(realpath "$SCRIPT_DIR/../../")"`
- `BASE_DIR="$PROJECT_ROOT/PKI"`
- `INT_CA_DIR="$BASE_DIR/intermediate"`
- `USER_DIR="$BASE_DIR/users/users/$USER_NAME"`
-
- `KEY_FILE="$USER_DIR/$USER_NAME.key.pem"`
- `CSR_FILE="$USER_DIR/$USER_NAME.csr.renovado.pem"`
- `NEW_CERT_FILE="$USER_DIR/$USER_NAME.cert.renovado.pem"`
- `P12_FILE="$USER_DIR/$USER_NAME.renovado.p12"`

Descrição:

- Chave original do user
- Novo CSR
- Novo certificado
- Novo .p12

2. Gerar novo CSR com a mesma chave

- `openssl req -new -key "$KEY_FILE" -out "$CSR_FILE"`

Descrição:

- Cria um novo CSR usando a chave antiga.
- O utilizador não precisa gerar uma nova chave para renovar.

3. Assinar novo certificado com a CA Intermediate

- `openssl ca -batch -config "$INT_CA_DIR/openssl.cnf" \`
- `-extensions usr_cert \`
- `-days 375 -notext -md sha256 \`
- `-in "$CSR_FILE" \`
- `-out "$NEW_CERT_FILE"`

4. Emitir um novo certificado

- openssl ca -batch -config "\$INT_CA_DIR/openssl.cnf" \
- -extensions usr_cert \
- -days 375 -notext -md sha256 \
- -in "\$CSR_FILE" \
- -out "\$NEW_CERT_FILE"

Descrição:

- A CA Intermédia assina o novo CSR.
- Gera um novo certificado com validade de 375 dias.

5. Criar novo .p12

- openssl pkcs12 -export \
- -inkey "\$KEY_FILE" \
- -in "\$NEW_CERT_FILE" \
- -certfile "\$INT_CA_DIR/certs/ca-chain.cert.pem" \
- -out "\$P12_FILE"

Descrição:

- Combina a chave privada, o novo certificado e a cadeia de confiança.
- Gera o ficheiro .p12, pronto a importar no browser ou usar com curl.

Reemitir um Certificado de um Utilizador:

reissue_revoked_users.sh

Reemitir um certificado significa emitir um novo certificado para um utilizador que já tinha um, mas cujo certificado foi revogado.

Por norma um certificado é reemitido quando o utilizador comprometeu ou perdeu o certificado anterior, certificado foi revogado por engano ou pretende-se continuar a usar a mesma identidade sem alterar o user.

Este script implementa os seguintes passos:

1. Verificar se o certificado foi mesmo revogado

- `SERIAL_HEX=$(openssl x509 -in "$OLD_CERT_FILE" -noout -serial | cut -d= -f2)`
- `IS_REVOKED=$(openssl crl -in "$INT_CA_DIR/crl/intermediate.crl.pem" -text -noout | grep "$SERIAL_HEX")`

Descrição:

- Obtém o número de série do certificado
- Verifica se ele aparece na CRL atual
- Se não estiver revogado, o script aborta — deve-se usar o script de renovação, não o de reemissão

2. Backup do certificado anterior

- `mv "$OLD_CERT_FILE" "$USER_DIR/${USER_NAME}.cert.revogado.pem"`

Descrição:

- Guarda o certificado revogado com um novo nome
- Evita sobrescrever acidentalmente

3. Gerar novo CSR (reutilizando a chave)

- `openssl req -new -key "$KEY_FILE" -out "$NEW_CSR_FILE"`

4. Emissão do novo certificado

- `openssl ca -batch -config "$INT_CA_DIR/openssl.cnf" \`
- `-extensions usr_cert \`
- `-days 375 -notext -md sha256 \`
- `-in "$NEW_CSR_FILE" \`
- `-out "$NEW_CERT_FILE"`

Descrição:

- A CA Intermédia emite um novo certificado como se fosse “de raiz”
- Usa as mesmas extensões de cliente (clientAuth, etc.)

5. Gerar novo .p12

- cp "\$NEW_CERT_FILE" "\$USER_DIR/\$USER_NAME.cert.pem"
- cp "\$NEW_P12_FILE" "\$USER_DIR/\$USER_NAME.p12"

Descrição:

- Junta o novo certificado, a chave privada e a cadeia de confiança
- Permite ao utilizador importar diretamente no navegador

6. Substituir ficheiros principais

- cp "\$NEW_CERT_FILE" "\$USER_DIR/\$USER_NAME.cert.pem"
- cp "\$NEW_P12_FILE" "\$USER_DIR/\$USER_NAME.p12"

Descrição:

- Substitui o certificado e .p12 antigos pelos novos

Referências

(What is PKI? A Public Key Infrastructure Definitive Guide | Keyfactor n.d.)

(wolfCrypt FIPS 140-2 and FIPS 140-3 | Licensing – wolfSSL n.d.)

(Welcome to Flask — Flask Documentation (3.1.x) n.d.)

(OpenSSL Essentials: Working with SSL Certificates, Private Keys and CSRs | DigitalOcean n.d.)

(7. Hands-On 4: Generating and Revoking Your Own Certificates Using OpenSSL — RTI Security Plugins Getting Started 7.5.0 documentation n.d.)

(How to revoke an openssl certificate when you don't have the certificate - Stack Overflow n.d.)